

Estructura del proyecto Alimentación en Laravel

ÍNDICE

ÍNDICE.....	2
Resumen muy resumido.....	3
VIEWS.....	3
CSS.....	4
JS.....	4
IMG, TIPO DE TEXTO, LIBRERÍAS.....	4
CONFIG.....	4
CONTROLADOR.....	5
→ Ejemplo de uso GRÁFICAMENTE.....	6
NO TERMINADO.....	6

Resumen muy resumido

El proyecto está organizado por capas para reutilizar plantillas, separar estilos y mantener cada alimento independiente.

VIEWS

→ `/resources/views/layout/`

Contiene las plantillas base del proyecto.

◆ `plantilla.blade.php`

Incluye:

- Menú de navegación
- Contenido dinámico
- Footer

Sirve como estructura general para reutilizar el mismo diseño en todas las páginas.

→ `/resources/views/componentes/`

Contiene componentes reutilizables para las plantillas base.

◆ Footer

◆ Menú

◆ Pestañas

La idea es evitar repetir código en múltiples vistas.

→ `/resources/views/alimentos/`

Contiene toda la lógica visual relacionada con los alimentos.

◆ `home.blade.php`

- Página inicial del proyecto.

◆ `plantilla-alimentos.blade.php`

- Plantilla general utilizada para mostrar cualquiera de los 6 alimentos.

En lugar de crear una página completa para cada alimento, se reutiliza esta plantilla y se le carga contenido dinámicamente.

→ Carpetas de cada alimento

Ejemplo:

◆ `/resources/views/alimentos/aceite/`

◆ `/resources/views/alimentos/huevo/`

◆ `/resources/views/alimentos/leche/`

Cada carpeta contiene las distintas secciones/fragmentos utilizados por la plantilla general.

Ejemplos:

- introducción
- valor-nutricional
- curiosidades
- metodos
- etc.

De esta forma:

- Cada alimento queda separado
- Es más fácil mantener contenido
- Se reutiliza la misma plantilla general

CSS

→ /public/css/

Contiene estilos globales del proyecto:

- ◆ Plantilla base
- ◆ Home
- ◆ Estilos generales

→ /public/css/alimentos/

- ◆ Contiene estilos relacionados con los alimentos.
 - Es la paleta de colores de cada alimento

→ /public/css/global/

- ◆ Archivos compartidos entre todos los alimentos.

→ /public/css/pestannas/

- ◆ Todos los alimentos tendrán la misma estructura, por tanto, contiene los estilos de cada pestaña
 - Para los colores se usa variables como var(--color-x)

JS

→ /public/js/

- ◆ Scripts generales de la plantilla principal

Ejemplo:

- ◆ toggle de navegación para menú desplegable

→ /public/js/alimentos/

- ◆ Scripts específicos para cada alimento.
- ◆ Cada alimento puede tener su propia lógica JS independiente.

→ /public/js/global/

- ◆ Scripts que usan todos los alimentos

IMG, TIPO DE TEXTO, LIBRERÍAS

→ /public/font/

- ◆ Fuentes utilizadas en el proyecto.

→ /public/img/

- ◆ Contiene carpetas de cada alimentos con sus imágenes

→ /public/img/svg/

- ◆ SVG utilizados para iconos y elementos visuales.

Ejemplos:

- ◆ huevo
- ◆ home
- ◆ user
- ◆ iconos decorativos

→ /public/jquery/

- ◆ Archivos relacionados con jQuery.

CONFIG

→ /config/alimentos.php

- ◆ Archivo principal de configuración dinámica de los alimentos.
- ◆ Aquí se define toda la información que necesita la plantilla para construir cada página:
 - introducción
 - css
 - secciones
 - js
 - títulos
 - vistas

Ejemplo:

```
'aceite' => [  
  'intro' => '...',  
  'css' => [...],  
  'secciones' => [...],  
  'js' => [...],  
]
```

- ◆ Gracias a esta configuración:
 - no hace falta crear controladores distintos por alimento
 - la plantilla carga contenido dinámicamente
 - es más sencillo añadir nuevos alimentos

CONTROLADOR

→ /app/Http/Controllers/AlimentoController.php

- ◆ Controlador encargado de cargar la información dinámica de los alimentos.

Funciones principales:

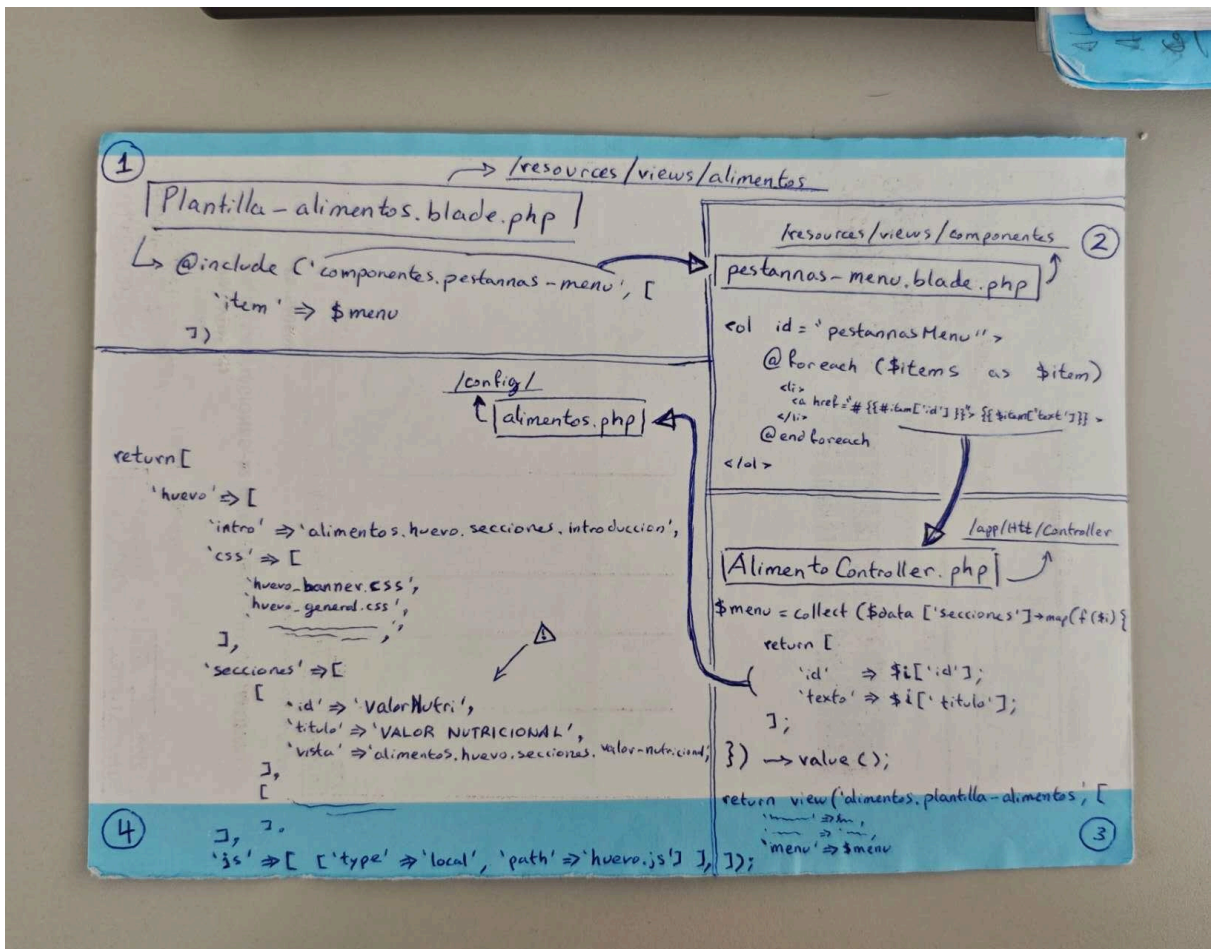
- ◆ Obtener datos desde config/alimentos.php
- ◆ Verificar si el alimento existe
- ◆ Generar el menú dinámicamente
- ◆ Enviar toda la información a la vista

La vista utilizada es:

- ◆ alimentos.plantilla-alimentos

Esto permite reutilizar una única plantilla para todos los alimentos.

→ Ejemplo de uso GRÁFICAMENTE



NO TERMINADO

→ `/resources/views/auth/`

◆ Carpeta reservada para futuras funcionalidades de autenticación/login.